```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import random
```

```
text = """
artificial intelligence is transforming modern society.
it is used in healthcare finance education and transportation.
machine learning allows systems to improve automatically with experience.
data plays a critical role in training intelligent systems.
large datasets help models learn complex patterns.
deep learning uses multi layer neural networks.
neural networks are inspired by biological neurons.
each neuron processes input and produces an output.
training a neural network requires optimization techniques.
gradient descent minimizes the loss function.

natural language processing helps computers understand human language.
text generation is a key task in nlp.
language models predict the next word or character.
recurrent neural networks handle sequential data.
lstm and gru models address long term dependency problems.
however rnn based models are slow for long sequences.

transformer models changed the field of nlp.
they rely on self attention mechanisms.
attention allows the model to focus on relevant context.
transformers process data in parallel.
this makes training faster and more efficient.
modern language models are based on transformers.

education is being improved using artificial intelligence.
intelligent tutoring systems personalize learning.
automated grading saves time for teachers.
online education platforms use recommendation systems.
technology enhances the quality of learning experiences.

ethical considerations are important in artificial intelligence.
fairness transparency and accountability must be ensured.
ai systems should be designed responsibly.
data privacy and security are major concerns.
researchers continue to improve ai safety.

text generation models can create stories poems and articles.
they are used in chatbots virtual assistants and content creation.
generated text should be meaningful and coherent.
evaluation of text generation is challenging.
human judgement is often required.

continuous learning is essential in the field of ai.
research and innovation drive technological progress.
students should build strong foundations in mathematics.
programming skills are important for ai engineers.
practical experimentation enhances understanding.
"""
```

```
chars = sorted(list(set(text)))
vocab_size = len(chars)

stoi = {ch:i for i,ch in enumerate(chars)}
itos = {i:ch for i,ch in enumerate(chars)}

def encode(s):
    return [stoi[c] for c in s]

def decode(l):
    return ''.join([itos[i] for i in l])

data = torch.tensor(encode(text), dtype=torch.long)
```

```
block_size = 40

def get_batch(batch_size=32):
    ix = torch.randint(len(data)-block_size, (batch_size,))
```

```
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    return x, y
```

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-np.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + self.pe[:, :x.size(1)]
        return x
```

```
class TransformerLM(nn.Module):
    def __init__(self, vocab_size, d_model=128, nhead=4, num_layers=2):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=256,
            dropout=0.2,
            batch_first=True
        )

        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers)
        self.fc = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoder(x)
        x = self.transformer(x)
        x = self.fc(x)
        return x
```

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = TransformerLM(vocab_size).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.003)
criterion = nn.CrossEntropyLoss()
```

```
epochs = 1500

for epoch in range(epochs):
    x, y = get_batch()
    x, y = x.to(device), y.to(device)

    logits = model(x)
    loss = criterion(logits.view(-1, vocab_size), y.view(-1))

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

```
Epoch 0, Loss: 0.0998
Epoch 100, Loss: 0.0483
Epoch 200, Loss: 0.0416
Epoch 300, Loss: 0.0382
Epoch 400, Loss: 0.0399
Epoch 500, Loss: 0.0390
Epoch 600, Loss: 0.0398
Epoch 700, Loss: 0.0377
```

```
Epoch 800, Loss: 0.0261
Epoch 900, Loss: 0.0256
Epoch 1000, Loss: 0.0459
Epoch 1100, Loss: 0.0303
Epoch 1200, Loss: 0.0158
Epoch 1300, Loss: 0.0327
Epoch 1400, Loss: 0.0334
```

```
def generate_text(model, start_text="artificial ", length=200):
    model.eval()

    input_ids = torch.tensor(encode(start_text), dtype=torch.long).unsqueeze(0).to(device)

    for _ in range(length):
        logits = model(input_ids)
        next_token_logits = logits[0, -1]
        probs = torch.softmax(next_token_logits, dim=0)

        next_id = torch.multinomial(probs, 1).item()
        input_ids = torch.cat([input_ids, torch.tensor([[next_id]]).to(device)], dim=1)

    return decode(input_ids[0].tolist())

print(generate_text(model))
```

```
artificial                              n f  n in n n in na i i nal l ini l l  a i ial a a ifical  al a iccral l a a a a aic
```